COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate route for games

04

Worked answers and solutions

Separate review edition

Original and application answers, exercise solutions, transfer review and C-bridge answers.

PACKAGE 04 / EXPANDED VOLUME 03 / SEPTEMBER 28, 2026
Independent study, implementation, evidence and review.

Contents

Worked answers and review guide 3

Module 11 — Native programming with C++ 4

Module 12 — Ownership, abstractions and C++ design 10

Module 13 — Data structures and their costs 16

Module 14 — Algorithms for maps, simulation and tools 21

Module 15 — Programming languages, compilers and computability 27

C-bridge answer key 32

Sources and provenance 33

Worked answers and review guide

Keep this separate from the first attempt

The answer key is for checking reasoning after doing the work. For each original lesson question, score0 for absent/incorrect,1 for partial and2 for correct with explanation; preserve the original5/6 formative threshold. With the two new application questions, a suggested expansion threshold is8/10 while retaining5/6 on the original core. Practical work and assigned study are still required.

Answers1–3 below are preserved verbatim from the original curriculum. Answers4–5, the40 exercise solutions,25 module reviews,20 transfer-review notes and12 C-bridge answers are additions. A note explaining a design is not a substitute for student code, tests or a fresh-build demonstration.

Module 11 — Native programming with C++

Lesson 11.1 — answers

11.1.Q1 — What task does the linker perform?

It resolves symbol references and combines object code/libraries into the requested output.

11.1.Q2 — Can valid declarations alone supply every function implementation?

No. A referenced function generally needs a matching definition unless special language rules apply.

11.1.Q3 — Why record build configuration with a benchmark?

Optimization, instrumentation and assertions can materially change behavior and performance.

11.1.Q4 — A build reports failure, but an executable still exists. Is its output evidence that your edit works?

No. It may be an old artifact. Check the command exit status, rebuild successfully and identify the executable from that build.

11.1.Q5 — Why does an include guard not solve ordinary duplicate function definitions in two translation units?

It suppresses repeated header text only within each translation unit. Both units can still define the function; place the ordinary definition in one source or use an appropriate inline definition.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_11_1.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: ordinary input 0 -> 1; ordinary input 41 -> 42; maximum int -> overflow error before addition; missing declaration -> compile rejection; missing definition -> link rejection.

Transfer reasoning: The declaration and definition describe different overloads. Make their signatures identical in the shared header and implementation, include that header in both callers/ implementation, then relink the defining object. A call-site cast does not supply a missing definition.

Lesson 11.2 — answers

11.2.Q1 — How many distinct bit patterns fit in eight bits?

1.

11.2.Q2 — Is ordinary signed integer overflow defined to wrap in C++?

No. It is undefined behavior; use checked arithmetic or an intentionally suitable representation.

11.2.Q3 — Why is writing raw struct bytes not a portable format?

Padding, byte order, alignment, type widths and representation may vary.

11.2.Q4 — Why is `if (a + b > max)` not a safe signed-overflow check?

The addition itself may already have undefined behavior. Rearrange the bound to validate operands before computing the sum.

11.2.Q5 — Can a serializer and deserializer round-trip correctly yet disagree with the file specification?

Yes. They can share the same mistake. Compare against independently specified bytes in addition to round-trip tests.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_11_2.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: 0 and exact application maximum accepted; negative text and trailing junk rejected; one above the application limit rejected; huge decimal value rejected by parser; 42 records of 24 bytes accepted under 1024; 43 rejected; 0x12345678 encodes to 78 56 34 12.

Transfer reasoning: Validate each dimension and bytes-per-pixel in their original domains. Use checked division against the allocation budget before multiplying, or a proven sufficiently wide intermediate with an explicit final budget check. A10^10-pixel request should be rejected by the chosen budget, not allowed to wrap into a small allocation.

Lesson 11.3 — answers

11.3.Q1 — Does a raw pointer necessarily own its target?

No. Ownership must be established by the interface or an owning type.

11.3.Q2 — Why is returning a reference to a local variable invalid?

The local object is destroyed at function exit, leaving a dangling reference.

11.3.Q3 — Does const by itself make shared access thread-safe?

No. Thread safety depends on all accesses, object behavior and synchronization.

11.3.Q4 — A `std::span` is copied into a job that runs after its vector is destroyed. Is the copy safe?

No. Copying the view does not copy or own the elements. Keep the owner alive or copy the required data into owned job state.

11.3.Q5 — Why might returning a small position by value be better than returning a reference?

It avoids coupling the caller to the source lifetime and invalidation rules, at the cost of a small explicit copy.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_11_3.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: empty range -> absent; present ID -> copied expected position; missing ID -> absent; null optional target -> explicit result; copied result remains valid after source growth; dangling local identified by inspection.

Transfer reasoning: Capture an owned copy of the required value, or capture a suitable owning shared object when a real shared lifetime is justified. A weak handle/ID with validity checking is another borrow design. Capturing by reference is safe only when the referenced owner is guaranteed to outlive execution.

Lesson 11.4 — answers

11.4.Q1 — Does reserve(100) make a vector's size 100?

No. It changes capacity, not the number of constructed elements.

11.4.Q2 — Why can a pointer to a vector element become invalid after growth?

Reallocation may move elements to a different storage region.

11.4.Q3 — What is wrong with ignoring every exception?

It hides failures and may allow incorrect or partially updated state to appear successful.

11.4.Q4 — Can vector erase invalidate a reference when capacity stays constant?

Yes. References and iterators at or after the erased position are invalidated even without reallocation.

11.4.Q5 — When is an optional copied value a suitable lookup result?

When absence is expected and a small independent snapshot satisfies the caller, avoiding an exported borrowing lifetime.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_11_4.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: empty lookup -> absent; duplicate ID -> rejected with unchanged size; growth preserves logical ID lookup; missing removal -> false and unchanged state; erase adjacent zero-HP units correctly; reserve changes capacity, not constructed count.

Transfer reasoning: Finish mutations before constructing a copied render snapshot, or rebuild short-lived borrows after mutation and guarantee no invalidating mutation until consumption completes. Asynchronous rendering needs ownership or explicit completion lifetime. Keeping old vector iterators is not repaired by merely reserving capacity.

Module 11 — exercise solutions

E11.1 — Trace the build graph

Problem: A game executable consists of main.cpp, world.cpp and world.hpp. main includes world.hpp; world.cpp supplies update_world. Draw the preprocessing, compilation and linking stages. Identify what must rebuild after a public signature changes.

Worked reasoning: Both translation units that include the changed header must be recompiled, then the executable relinked. A dependency-aware generator tracks that header relationship. The header is not normally compiled into a standalone implementation. Preserve the declaration/definition match and show the actual verbose build, rather than claiming that a filename proves a stage occurred.

E11.2 — Define a checked budget

Problem: A texture header declares width and height as decimal unsigned values, each at most 16,384. Pixels need four bytes and the allocation budget is 64 MiB. Specify a checked validation order and give one legal and one rejected case.

Worked reasoning: Parse each dimension with complete-consumption and range checks; reject zero under this contract. Before multiplication, test width <= budget / 4 / height using nonzero divisors, or use an adequately bounded wider type and compare the result. 4096 by 4096 consumes exactly 67,108,864 bytes and fits; 4097 by 4096 does not. State that 1 MiB is 1,048,576 bytes. A cast after an overflowing multiplication cannot repair it.

E11.3 — Repair a borrowed view

Problem: A function builds a local std::string and returns std::string_view into it. Identify the owner, the lifetime end and two valid redesigns. Do not execute the defect to prove it.

Worked reasoning: The local string owns the characters and is destroyed on return. The view does not extend that storage lifetime. Return an owning string by value, or accept caller-owned storage and return a view whose documented lifetime is bounded by that storage and invalidating mutations. A static scratch buffer is not an equivalent general repair: it changes sharing, reentrancy and thread behavior.

E11.4 — Choose stable identity

Problem: You keep a pointer to an element of a vector of units. More units spawn, then an earlier element is erased. Design a lookup contract that does not depend on old addresses.

Worked reasoning: Use a stable logical ID and resolve it when needed; return a copied snapshot or a short-lived borrowed reference with explicit invalidation rules. Vector growth can reallocate, and erase invalidates references/iterators at and after the erase position. reserve can prevent some reallocations but not erase-related invalidation or ownership destruction. If IDs are reused, include a generation or document a no-reuse policy.

E11.5 — Binary encoding

Problem: Encode entity ID 0x01020304 as four little-endian bytes. Explain why writing sizeof(Unit) bytes from an object is not this format.

Worked reasoning: Bytes are 04 03 02 01. Native object layout can include padding, and native byte order need not equal the specified wire order. Pointers and library containers are not portable serialized payloads. Write named fields with checked widths and lengths, and test the exact bytes against a golden fixture.

E11.6 — Floating-point evidence

Problem: On your compiler, print max_digits10, digits and is_iec559 for float and double. Compare (a+b)+c with a+(b+c) for a large cancellation case. What is the conclusion allowed by one run?

Worked reasoning: Record actual properties and flags. A suitable demonstration is a=1e16, b=-1e16, c=1 in common binary64 arithmetic: the first grouping often yields 1 and the second 0. Treat the observed result as a property of that build and rounding behavior, not every C++ platform. The reason is rounding at intermediate operations; associativity over real numbers does not automatically transfer to floating point. Do not enable fast-math for the baseline.

E11.7 — Error contracts

Problem: A resource lookup may legitimately miss, a packet may be malformed, and a developer may violate an internal invariant. Design distinct observable outcomes and tests for each.

Worked reasoning: An optional value suits ordinary absence. A structured parse error with category and offset suits malformed input. An invariant violation needs a deliberate fail-fast or checked error policy, not silent recovery that corrupts state. The particular mechanism can differ, but callers must be able to distinguish the cases; no stale output may masquerade as success.

E11.8 — Cross-language replay

Problem: Define a six-command, integer-only grid replay and run equivalent Python and C++ transitions. Include an edge hit and a no-op. Compare complete states, not just a printed final coordinate.

Worked reasoning: Specify initial state, grid bounds, legal deltas, collision policy, command ordering and tick advancement before coding. Compare each indexed state and whether each command was accepted. An independent Python fixture is supplied in code/tools/grid_fixture.py. Agreement only covers the stated model: it does not establish floating-point determinism, RNG portability or equivalence to the learner's entire earlier game.

Module 11 — review answers

M11.R1 — Explain compile, link and runtime failure using one concrete defect for each.

Syntax or missing declaration is compile-time; declared but omitted definition is link-time; a rejected out-of-bounds command is a runtime contract failure. Include actual diagnostic evidence.

M11.R2 — Why is cast-then-check too late for a large file length?

The cast can discard the original range information. Validate the original value and the multiplication/allocation budget before narrowing or allocating.

M11.R3 — When is a returned span safe?

Only while its referenced storage remains alive and access is not invalidated; span does not own or extend lifetime. State mutation rules.

M11.R4 — What is the difference between an entity ID and a vector index?

An ID names a logical entity; an index names a current position that can change with insertion/erase/reordering. Reuse policies need explicit handling.

M11.R5 — What evidence establishes the native port rather than just successful compilation?

Equivalent specified state transitions and error policy on shared fixtures, automated tests, explicit representation choices and a clean reproducible build.

Module 12 — Ownership, abstractions and C++ design

Lesson 12.1 — answers

12.1.Q1 — What does RAII connect?

Resource ownership and release to the lifetime of an object.

12.1.Q2 — Can shared ownership create a leak through cycles?

Yes, if owning references keep each other alive; weak/non-owning links may be needed.

12.1.Q3 — Why must device-resource destruction order be planned?

A release operation may require a live device or context; destroying the device first invalidates that assumption.

12.1.Q4 — Does an enclosing object whose constructor throws get its normal destructor call?

No. Fully constructed bases and members are destroyed, but the incomplete enclosing object is not destroyed as a fully constructed object.

12.1.Q5 — Why can correct RAII still coexist with a use-after-free bug?

Another object can retain a non-owning borrow after the owner is destroyed. Release ownership and borrow lifetime are related but distinct obligations.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_12_1.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: successful scope releases exactly once; injected exception releases exactly once; empty owner releases nothing; nested owners unwind in reverse order; all paths return live count to baseline.

Transfer reasoning: Provide an explicit close/flush operation that reports errors, then make destructor cleanup best-effort and nonthrowing. Track whether the handle has already been released, and document state after a failed flush/close. The destructor must not perform a second release after a completed explicit close.

Lesson 12.2 — answers

12.2.Q1 — Does std::move itself guarantee that an object is moved?

No. It permits move-aware overload selection; the selected operation determines what happens.

12.2.Q2 — What is the strong exception guarantee?

On failure, the operation leaves the relevant observable state unchanged.

12.2.Q3 — Why prefer the rule of zero?

Correct member types can manage resources without fragile custom copy/move/destruction code.

12.2.Q4 — Why might transactional replacement need more peak memory than destructive replacement?

The old asset is retained while the new candidate is prepared so failure can leave the old state intact.

12.2.Q5 — What happens when an exception escapes a function declared `noexcept`?

The program calls termination rather than propagating the exception normally; the guarantee must reflect the implementation.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_12_2.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: copy constructible is false; move leaves exactly one owning handle; move assignment releases prior destination; self-move remains valid; load/validation failure preserves old asset; successful replacement releases old asset once.

Transfer reasoning: The asset and index form one observable repository invariant. Prepare both in candidate state and commit them together through a nonthrowing mechanism, or provide rollback with a justified guarantee. Swapping the asset and then failing while updating the index exposes a partially changed repository.

Lesson 12.3 — answers

12.3.Q1 — What does substitutability require?

An implementation used through an interface must preserve that interface's behavioral contract.

12.3.Q2 — Why can a polymorphic base need a virtual destructor?

Deletion through a base pointer must invoke appropriate derived cleanup.

12.3.Q3 — Why not put every responsibility into one engine class?

It increases coupling, obscures contracts and makes isolated testing or replacement difficult.

12.3.Q4 — A backend retains a span although the interface promises use only during the call. Is matching output today sufficient?

No. It violates the lifetime contract and can fail after the source storage changes or is destroyed.

12.3.Q5 — Why might one virtual call per batch have a different cost from one per entity?

The dispatch cost is amortized over different amounts of work and the loop/inlining/locality opportunities differ; measure the actual workload.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_12_3.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: empty input emits no commands; two units preserve input order; recorded commands own copied data; derived cleanup observed through base ownership; simulation does not mutate renderer ownership.

Transfer reasoning: Queue owned command payloads or transfer an owning batch whose lifetime extends through completion. Alternatively enforce a documented completion barrier before caller storage can change or die. A reference to a temporary DrawCommand does not become valid simply because submission returned successfully.

Lesson 12.4 — answers

12.4.Q1 — What does a concept constrain?

The compile-time requirements a type or expression must satisfy for a template interface.

12.4.Q2 — Is structure-of-arrays universally superior?

No. Its benefit depends on access pattern, workload and architecture.

12.4.Q3 — Name a cost of templates beyond runtime.

Compilation time, error complexity, binary size or maintenance burden.

12.4.Q4 — Does satisfying a concept prove runtime bounds are valid?

No. Concepts constrain compile-time requirements; runtime domain conditions still need validation or a documented precondition.

12.4.Q5 — Why must both layout variants produce an observable checked result?

It verifies equivalent work and prevents the benchmark from timing a computation the optimizer can discard as unused.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_12_4.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: below, within and above bounds; equal lower and upper bounds; reversed bounds rejected; same checksum for both layouts; zero entities handled; multiple sizes and repeated samples recorded.

Transfer reasoning: Fetching all fields of one entity can favor contiguous per-entity AoS storage, whereas scanning one field across entities can favor SoA. Compare equivalent selected-entity access patterns, sizes, order and checksums in Release; publish measured results rather than treating either layout as universally superior.

Module 12 — exercise solutions

E12.1 — Partial construction

Problem: An object owns resource A and then tries to acquire B, which fails. Draw which fully constructed subobjects are destroyed and whether the outer object destructor runs.

Worked reasoning: Fully constructed members and bases are destroyed during unwinding in reverse completion order; the destructor of the incompletely constructed outer object does not run. Put A in an owning member rather than an unguarded raw handle. Count acquisitions and releases to show that A is released even though construction never returned an object.

E12.2 — Move assignment

Problem: An owner contains handle 7; a second owner contains handle 9. Move-assign the second into the first. State the destination, source and release counts. Include self-move in the contract.

Worked reasoning: The old destination resource 7 must be released; the destination becomes the unique owner of 9; the source becomes empty under this wrapper's contract. Destruction then releases 9 once. The educational wrapper treats self-move as a no-op. Standard-library moved-from objects generally have valid but otherwise unspecified states unless a stronger guarantee is specified; do not infer universal emptiness.

E12.3 — Transactional replacement

Problem: A loaded level contains a texture handle and parsed metadata. A replacement may fail during file reading, validation or allocation. Describe the commit point.

Worked reasoning: Prepare a complete candidate level in local owning objects, validate it, then swap or move through an operation with a suitable nonthrowing guarantee. Failures before commit destroy candidate resources and preserve the old level. A failure after destroying the old level cannot satisfy the strong guarantee. The exercise must inject failures at multiple acquisition stages rather than only throw before all work.

E12.4 — Shared ownership cycle

Problem: Two shared_ptr-owning nodes point to one another. External roots are removed. Explain the leak mechanism and draw a corrected ownership graph.

Worked reasoning: Each strong reference keeps the other count nonzero, so both objects remain owned after external roots disappear. Choose a real ownership direction and make back references weak_ptr or non-owning IDs with a lifetime contract. A weak reference must be locked and checked before use. shared_ptr is not a tracing garbage collector and does not solve every graph ownership problem.

E12.5 — Substitutability

Problem: A base renderer promises that draw captures a command for later execution. One subclass retains a reference to a caller's temporary command instead. Does virtual dispatch make the implementation correct?

Worked reasoning: No. The implementation violates the promised lifetime behavior. Store an owned copy or transfer ownership under a different explicit interface. Virtual functions select an implementation; they do not prove that the implementation preserves postconditions, error behavior or lifetime requirements.

E12.6 — Concept versus invariant

Problem: Your template requires that a value type supports <. Does that establish that a comparator is a strict weak ordering? Give a counterexample to purely syntactic checking.

Worked reasoning: No. A requires-expression checks whether expressions are well formed and perhaps their result types, not all semantic laws. A comparator that returns true for equal values can compile while violating irreflexivity. State and test irreflexivity, asymmetry and transitivity on representative values, while recognizing that finite tests are not a universal proof.

E12.7 — Layout experiment

Problem: Compare an array of structs with a struct of arrays for an update using only x and vx. Define a fair benchmark and a reason the result might change for another workload.

Worked reasoning: Use identical entity counts, values, updates and correctness checks; exclude initialization/allocation consistently; alternate order; repeat timed samples in Release without sanitizers; report hardware and dispersion. AoS may pull unused fields into cache, but an operation using all fields changes the traffic pattern. Do not claim that one synthetic run establishes a universal layout winner.

E12.8 — One-page ownership map

Problem: Draw a resource manager with a unique owner, a cache, a renderer and a temporary command list. Label every edge owning or borrowing and each invalidating action.

Worked reasoning: A valid answer names a single authoritative owner or a justified shared lifetime for each resource. It identifies destruction order, cache eviction, generation checks or expired weak references, and whether commands copy data or hold handles. The map must explain replacement failure and renderer shutdown. Review against code, because an elegant diagram cannot compensate for an untracked raw owner.

Module 12 — review answers

M12.R1 — Why should a resource be placed into an owner immediately?

The owner then covers later failure paths automatically; an unguarded raw handle between acquisition and adoption can leak.

M12.R2 — Does std::move itself move the resource?

No. It changes value category to permit move operations; the selected constructor or assignment defines the actual behavior.

M12.R3 — When can replacing an object satisfy the strong guarantee?

Prepare a complete valid candidate while preserving the original, then commit through a suitable nonthrowing operation.

M12.R4 — Why is a virtual destructor relevant to unique_ptr?

Deletion through the base pointer must correctly destroy the dynamic object when that polymorphic ownership contract is used.

M12.R5 — Why are layout benchmark results conditional?

They depend on accessed fields, working set, compiler, optimization, machine, order and workload; publish those conditions and repeated samples.

Module 13 — Data structures and their costs

Lesson 13.1 — answers

13.1.Q1 — Does O(n) specify an exact runtime?

No. It bounds growth under a cost model and omits constants and environment details.

13.1.Q2 — What is the total 1+...+(n-1) proportional to?

n squared, since the sum is n(n-1)/2.

13.1.Q3 — How does amortized analysis differ from random-input average-case analysis?

It bounds total cost over an operation sequence without requiring a probability distribution over inputs.

13.1.Q4 — Can an amortized O(1) append take O(n) time on one call?

Yes. A growth call can relocate n elements while total cost over the sequence remains linear.

13.1.Q5 — Why does timing two small input sizes not prove an O(n log n) bound?

Finite noisy measurements do not establish the operation-growth proof and may be dominated by constants or different regimes.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_13_1.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: empty buffer has size 0; out-of-range access rejected; n=8 exact-fit moves 28; doubling moves 7; n=9 doubling moves 15; values preserved through growth; capacity limit checked before arithmetic.

Transfer reasoning: Amortized bounds allow occasional expensive allocation/copy spikes. Reserve a justified maximum before the frame phase, use fixed-capacity storage or a preallocated pool, and return an explicit full/overflow result when the supported budget is exhausted. Test the failure without reallocating during the protected phase.

Lesson 13.2 — answers

13.2.Q1 — Which item leaves first after pushing A then B onto a stack?

B.

13.2.Q2 — Why does a ring buffer need a full/empty convention?

Equal index values can otherwise be ambiguous after wrapping.

13.2.Q3 — Does a correct single-threaded queue imply a thread-safe queue?

No. Concurrent access introduces ordering, atomicity and ownership requirements.

13.2.Q4 — After wrapping, must the front item be stored at array index zero?

No. The head index identifies the logical front, and FIFO order can cross the end of the array.

13.2.Q5 — Why must a failed push leave existing entries unchanged?

The explicit reject-on-full contract promises not to lose or alter already queued commands.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_13_2.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: pop empty -> absent; push full -> false with unchanged state; capacity-one alternating operations; wraparound returns FIFO order; more than 3*capacity operations; no read from a logically empty slot.

Transfer reasoning: An overwrite-oldest policy evicts the logical head when full before inserting the new sample. Update full-state return semantics, head/count invariants and expected pop sequence. Real audio concurrency and timing requirements need separate synchronization analysis; changing overflow semantics alone does not create a lock-free queue.

Lesson 13.3 — answers

13.3.Q1 — Does a matching hash prove two keys are equal?

No. Collisions are possible; equality must still be checked.

13.3.Q2 — What is the load factor for 30 entries in 10 buckets?

3 entries per bucket on average for this definition.

13.3.Q3 — Why is expected O(1) not an unconditional guarantee?

It depends on hashing, occupancy and input assumptions; worst cases can be much slower.

13.3.Q4 — Why can clearing an open-addressed slot to empty break lookup?

A later colliding key may lie beyond it; an empty marker can terminate the probe before that key is reached.

13.3.Q5 — Does load factor 1 guarantee every bucket contains exactly one entry?

No. It is an average. Distribution may be uneven, including many entries in one bucket and empty others.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_13_3.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: missing key -> absent; same key update does not increase size; distinct equal-hash keys preserved; erase chain head/middle/tail; all entries survive resize; negative and zero integer keys handled.

Transfer reasoning: Correctness can coexist with attacker-chosen quadratic work or excessive allocation. Production needs input-size/time/memory limits and a reviewed hashing/collision defense suitable to its threat model. This exercise does not certify a defense; identify worst-case resource behavior and fail within explicit budgets.

Lesson 13.4 — answers

13.4.Q1 — Does a heap provide a fully sorted array?

No. It guarantees the heap ordering relation, especially access to the extreme element.

13.4.Q2 — What can the height of an unbalanced n-node search tree become?

n in the chain-like worst case, depending on height convention by one.

13.4.Q3 — Why define a tie-break rule in a priority queue?

Equal priorities otherwise may produce inconsistent processing order, complicating tests and deterministic behavior.

13.4.Q4 — Is [2,7,5] a valid min-heap?

Yes. The root is no greater than either child; sibling order is unconstrained.

13.4.Q5 — Why is `a <= b` not a suitable less-than comparator?

It returns true for equal values, including comparing an item with itself, violating strict-order requirements.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_13_4.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: empty pop -> absent; one-element pop leaves empty; ascending and descending insertions; duplicate priorities retained; negative priorities ordered correctly; invariant after every randomized operation.

Transfer reasoning: A heap exposes the next priority efficiently but does not directly support ordered interval traversal. A balanced ordered tree or sorted structure may give O(log n+k) range reporting under its contract, at different insertion/storage costs. Define k as the output count and compare the actual update/query workload.

Module 13 — exercise solutions

E13.1 — Amortized growth

Problem: Starting empty, compare total old-element copies for capacity growth by exactly one and by doubling through eight insertions. Include the ninth insertion.

Worked reasoning: Exact growth copies 0+1+2+3+4+5+6+7=28 old elements by eight inserts and 36 by nine. Doubling capacities 1,2,4,8 copies 0+1+2+4=7 by eight and 15 by nine when growing to16. A single doubling push is linear in current size, while the aggregate is linear over the sequence. Allocation and value-write costs still exist.

E13.2 — A circular trace

Problem: A capacity-three reject-on-full queue receives push10, push20, pop, push30, push40, push50, pop, pop, pop. Write results, head/count changes and logical contents.

Worked reasoning: The pops yield10,20,30,40 in order; push50 is rejected and does not replace anything. With head initially0: after the first pop head1/count1; after push30 and40 count3; rejected50 leaves state unchanged; remaining pops advance head2,0,1 and count2,1,0. Physical array slots are not themselves the logical queue order.

E13.3 — Collision deletion

Problem: For open addressing, keys A and B hash to the same bucket and B is placed after A. Explain why replacing A with a never-used empty marker can break lookup.

Worked reasoning: Lookup may stop at the first never-used empty marker and falsely conclude B is absent. Use a tombstone or a correct backward-shift deletion policy consistent with the probe sequence. The supplied reference uses chaining instead, where deleting one node does not break another bucket-chain search; this problem is an independent policy comparison.

E13.4 — Hash-table adversary

Problem: A hash function returns zero for every key. What changes in expected versus worst-case lookup, and what should a correctness test demonstrate?

Worked reasoning: All keys collide. Chaining lookup becomes linear in the number of keys in that bucket, and a sequence of insertions/searches can become quadratic overall. The table must remain correct through updates, deletions and rehashes. Expected constant behavior relies on distribution/load assumptions; forced collision tests check correctness, not performance guarantees.

E13.5 — Heap does not mean sorted

Problem: Give a valid min-heap array that is not globally sorted. Explain the order guarantee and derive parent/child indices in a zero-based array.

Worked reasoning: [1,4,2,9,7,3] is a valid min-heap and not sorted. Every parent is <= its children; siblings and unrelated subtrees need not be ordered. Children are2i+1 and2i+2, parent is(i-1)/2 for i>0. Check bounds before using a child index and consider arithmetic limits for a generalized huge structure.

E13.6 — Tie semantics

Problem: Events have time and insertion sequence. Define a priority rule that produces reproducible FIFO behavior among equal times and test three ties.

Worked reasoning: Order by the pair(time,sequence), with the lower pair first; use a sequence domain that cannot silently wrap during the supported run. std::priority_queue's default comparator makes the largest element top, so a min-priority comparator needs the opposite relation. Distinct sequence values make same-time ordering explicit rather than dependent on heap layout.

E13.7 — Locality versus complexity

Problem: A linked list supports O(1) insertion after a known node. Why can a vector still win when inserting a small record into a small collection?

Worked reasoning: The list claim excludes finding the position, allocation and cache behavior. A vector may shift a small contiguous region efficiently and avoid per-node allocation/pointer chasing. Compare the complete operation and representative workload, not isolated asymptotic phrases. Iterator stability or large record movement may reverse the tradeoff.

E13.8 — Stateful differential test

Problem: Design a seeded randomized test for your map or queue against a standard container. Explain how to make a failure actionable.

Worked reasoning: Generate bounded operations and compare return values, contents, size and invariants after each step. Log seed, operation index and the complete or minimized failing prefix. Force empty/full/collision/boundary cases in addition to random sampling. A matching reference with the same bug is possible; retain hand-derived golden traces and invariant checks as separate evidence.

Module 13 — review answers

M13.R1 — Distinguish worst-case push from amortized push for a doubling array.

One growth push moves O(n) old elements; over a sequence of n pushes the total movement is O(n), giving amortized O(1) per push under the stated model.

M13.R2 — What happens on full in this ring buffer?

Push returns false and leaves contents, head and count unchanged. It is not an overwrite-oldest queue and not concurrent.

M13.R3 — What assumptions support expected constant hash lookup?

A suitable hash distribution and controlled load/collision policy. Worst-case collision chains can still be linear.

M13.R4 — Why can a heap not answer arbitrary membership queries in logarithmic time just from its ordering?

Only parent-child priority is ordered; an arbitrary target is not localized to a binary-search branch.

M13.R5 — What must every randomized container test record?

Reproducible seed, operation trace/failing prefix, expected/actual results and invariant checks, supplemented by boundary/golden cases.

Module 14 — Algorithms for maps, simulation and tools

Lesson 14.1 — answers

14.1.Q1 — What precondition does ordinary binary search require?

A range ordered consistently with its comparison rule.

14.1.Q2 — What does stable sorting preserve?

The original relative order of elements considered equivalent by the comparison.

14.1.Q3 — Why test element preservation as well as sortedness?

An output can be sorted yet incorrectly omit, duplicate or change input elements.

14.1.Q4 — What extra check turns a lower-bound position into a membership result?

Require the position to be within the range and the element to be equivalent to the target under the relevant comparison.

14.1.Q5 — Why test a sort on records containing original sequence IDs?

The IDs reveal whether equivalent-key elements kept their relative order, making stability observable.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_14_1.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: empty -> insertion position 0; target below all -> 0; duplicate target -> first equivalent position; target above all -> size; sorting preserves duplicate counts; equal-key records preserve order for stable implementation.

Transfer reasoning: Binary search requires the same consistent ordering/equivalence relation as sorting. Case-insensitive sort does not generally establish the case-sensitive partition precondition. Use the same comparator or rebuild under the required relation, including a defined treatment of equal folded names.

Lesson 14.2 — answers

14.2.Q1 — When does BFS find a shortest path by edge count?

On an unweighted graph, or when all traversed edges have equal cost.

14.2.Q2 — Can a directed graph with a cycle have a topological order?

No.

14.2.Q3 — Why mark visited nodes?

To avoid repeated exploration and nontermination on cycles while preserving the traversal's intended behavior.

14.2.Q4 — Why mark a BFS vertex discovered when enqueuing rather than repeatedly when dequeuing?

It prevents multiple frontier entries for the same newly discovered vertex and preserves the intended one-discovery invariant.

14.2.Q5 — How can you test a topological order without demanding one particular valid sequence?

Map vertices to output positions and check that every prerequisite-to-dependent edge goes from an earlier position to a later one, with every vertex appearing exactly once.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_14_2.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: source equals goal; isolated unreachable goal; diamond graph with two shortest paths; directed cycle; self-loop cycle; disconnected DAG includes all vertices; invalid vertex rejected.

Transfer reasoning: Each ready vertex is inserted/removed from a heap rather than a constant-time FIFO. With ordinary adjacency traversal and one insertion/removal per vertex, the bound is O(E+V log V), excluding separate validation/name-comparison costs. State string comparison costs when names are not constant-size.

Lesson 14.3 — answers

14.3.Q1 — What edge-cost condition does ordinary Dijkstra require?

Nonnegative edge costs.

14.3.Q2 — What is Manhattan distance from (0,0) to (3,4)?

1.

14.3.Q3 — Why can inconsistent heuristics require reopening nodes?

A better path to a previously processed node may be discovered later; permanently closing it can discard that improvement.

14.3.Q4 — For unit-cost diagonal and orthogonal moves on an unobstructed grid, why can Manhattan overestimate?

A diagonal reduces both coordinate differences for one unit of cost; max(|dx|,|dy|) is the appropriate geometric lower bound in that model.

14.3.Q5 — Why skip a priority-queue entry whose stored g differs from the current best distance?

It represents an obsolete route superseded by a better relaxation; processing it would add unnecessary or misleading work.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_14_3.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: start equals goal; unreachable region; weighted detour cheaper than direct route; zero-cost edge allowed; negative edge rejected; overestimating heuristic gives 10 versus optimal 2; A* with h=0 agrees with Dijkstra.

Transfer reasoning: With unit-cost orthogonal and diagonal moves, Chebyshev distance max(|dx|,|dy|) is a lower bound: one move can reduce each difference by at most one. Obstacles removing moves cannot shorten that minimum. Different diagonal costs require a corresponding revised lower bound.

Lesson 14.4 — answers

14.4.Q1 — What two features motivate dynamic programming?

Overlapping subproblems and a valid way to combine their solutions through sufficient state.

14.4.Q2 — Does NP-complete mean no instance can be solved quickly?

No. It concerns worst-case problem-family complexity, not every individual instance.

14.4.Q3 — Why keep a brute-force solver for tiny inputs?

It provides a simple correctness oracle for testing more sophisticated algorithms.

14.4.Q4 — Why must one-dimensional 0/1 knapsack update capacity downward?

It prevents the current item from being used again through a state already updated in the same item iteration.

14.4.Q5 — Why is O(nB) called pseudopolynomial when B is binary encoded?

The running time depends on B itself, which can be exponential in the number of bits used to encode B.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_14_4.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: no items -> 0; capacity zero -> 0; all items too heavy -> 0; (2,3),(3,4),(4,5), B=5 -> 7; single (2,3), B=4 -> 3, not 6; many deterministic small cases agree with exhaustive solver.

Transfer reasoning: A state needs the item prefix plus remaining/used mass and power budgets. A direct dense DP has O(nMP) time and O(MP) rolling-layer space for integer capacities M and P. This can exceed memory even when the one-budget problem is small; validate dimensions and consider a justified alternative.

Module 14 — exercise solutions

E14.1 — Half-open search proof

Problem: For lower_bound on a sorted array, state the loop invariant and show why lo+(hi-lo)/2 is preferable to (lo+hi)/2.

Worked reasoning: Maintain that positions before lo are below the target and positions at or after hi are not below it; the unresolved range is[lo,hi). Each update shrinks it, and lo==hi is the first possible position. The difference form avoids an unnecessary lo+hi overflow when indices are valid, while preserving the midpoint.

E14.2 — Stability as a requirement

Problem: Records (key,arrival) are (2,A),(1,B),(2,C),(1,D). Give a stable sorted result and one acceptable-looking but unstable result.

Worked reasoning: Stable order is(1,B),(1,D),(2,A),(2,C). Reversing B/D or A/C is unstable even though keys are sorted. If equal-key arrival order is part of the game or tool contract, test it explicitly. Otherwise adding a deterministic secondary key is another specified contract, not the same claim as stability.

E14.3 — Dependency direction

Problem: Texture T must exist before Material M, and M before Level L. Define edge direction, a valid topological order and a cycle rejection test.

Worked reasoning: Use prerequisite -> dependent edges T->M and M->L. A valid order is T,M,L. Adding L->T forms a cycle; a Kahn traversal that processes fewer than all vertices must reject the complete build order. Returning only an acyclic prefix as if it were success is incorrect.

E14.4 — BFS discovery rule

Problem: A diamond graph has S->A,S->B,A->G,B->G. Compare marking visited at enqueue versus after removal.

Worked reasoning: Marking on enqueue prevents G from being queued repeatedly and fixes the first shortest parent under fixed neighbor order. Marking late can enqueue duplicates, complicate parent semantics and increase work. BFS shortest-path claims require equal-cost edges; varying weights require another algorithm.

E14.5 — An inadmissible shortcut

Problem: S->G costs10, S->A costs1, A->G costs1. Set h(A)=100,h(G)=0. Trace goal-on-pop A*.

Worked reasoning: After expanding S, priorities are10 for G and101 for A, so G is popped with cost10 even though the optimum is2. The heuristic overestimates the remaining cost of A, violating admissibility. h=0 gives Dijkstra on this graph. A low expanded-node count does not repair the wrong result.

E14.6 — Movement changes the heuristic

Problem: Explain when Manhattan distance is appropriate for a grid and why introducing cheap diagonal moves can invalidate it.

Worked reasoning: For four-neighbor movement with each orthogonal step costing at least c>=0, c*(|dx|+|dy|) is a lower bound. If a diagonal moves both coordinates for the cost of one orthogonal move, Manhattan may overestimate: from(0,0)to(1,1) it predicts2c but a diagonal costs c. Use a lower bound derived from the actual movement and cost rules; do not reuse a heuristic merely because the map looks square.

E14.7 — 0/1 versus unbounded DP

Problem: One item weighs2 and is worth3; budget is4. Show how ascending in-place capacity iteration incorrectly permits reuse in a 0/1 solver.

Worked reasoning: Ascending updates first set dp2=3, then dp4 can read the updated dp2 and become6, using the same item twice. Descending capacity reads the previous-item layer implicitly and yields3. A two-dimensional table with separate item rows is another way to preserve that distinction.

E14.8 — Pseudo-polynomial cost

Problem: Your knapsack DP costs O(nB). Why is this not necessarily polynomial in the input bit length, and what should a real-time tool do with huge B?

Worked reasoning: The binary encoding length of B is proportional to log B, so iteration proportional to B can be exponential in that part of the encoded input. Enforce budgets, reduce the problem with a justified scaling/approximation policy, or choose another solver. State approximation error or lack of guarantee; silently truncating B changes the optimization problem.

Module 14 — review answers

M14.R1 — When does BFS provide minimum-cost paths?

When all traversed edges have the same nonnegative step cost, so minimizing edge count minimizes cost; the ordinary unit-cost case is simplest.

M14.R2 — What does a topological cycle failure mean for an asset build?

No ordering satisfies all dependencies. Report a cycle or failure; do not call the processed prefix a valid complete plan.

M14.R3 — Why skip stale priority-queue records in Dijkstra?

A node may be queued with an old larger tentative cost; expanding it after a better distance is known repeats work and may confuse termination/accounting.

M14.R4 — What does h=0 do to A*?

It prioritizes accumulated cost and gives Dijkstra behavior, subject to the same nonnegative-edge and implementation assumptions.

M14.R5 — What is required before claiming an approximation guarantee?

Define the objective/domain and prove or cite an applicable bound for that algorithm; an observed close result is not a general guarantee.

Module 15 — Programming languages, compilers and computability

Lesson 15.1 — answers

15.1.Q1 — Does static typing prove all logical requirements?

No. It enforces its type rules, which cover only part of correctness.

15.1.Q2 — What advantage does a pure calculation offer tests?

The same explicit inputs determine the result without hidden state or side effects.

15.1.Q3 — What must a C-compatible ownership boundary specify?

Who allocates and frees resources, valid lifetimes, error signaling and compatible representations.

15.1.Q4 — Can an interpreter perform static type checking before execution?

Yes. Static versus dynamic concerns when rules are checked, not whether execution uses an interpreter or machine code.

15.1.Q5 — Why must a C++ wrapper call the C API's destroy function rather than `delete`?

The API owns the allocation convention and release requirements. Mixing allocation families or bypassing API cleanup can corrupt memory or leak resources.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_15_1.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: damage below armor -> zero effective damage; damage exceeds remaining HP -> clamp at zero; negative domain input rejected; Number+Number accepted; Number+Boolean rejected; null output rejected without dereference.

Transfer reasoning: Pass the random sample, a deterministic RNG state or an explicit RNG dependency into the rule and return updated state as needed. Record the generator algorithm/ state and ordering. A seed alone is insufficient when library distribution algorithms or call order differ.

Lesson 15.2 — answers

15.2.Q1 — What separates lexing from parsing?

Lexing identifies tokens; parsing organizes tokens according to grammar.

15.2.Q2 — Under ordinary precedence, what is 2+3*4?

1.

15.2.Q3 — Why preserve source locations in a syntax tree?

They allow later semantic or runtime errors to point back to meaningful user code.

15.2.Q4 — Why must the parser require End after the expression?

Otherwise it can accept a valid prefix and silently ignore trailing malformed or unintended input.

15.2.Q5 — Why can an arena of indexed nodes be easier to own than pointers into a growable node vector?

Indices remain usable after vector reallocation when elements are not erased/reordered, whereas pointers into the old storage are invalidated; bounds still need validation.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_15_2.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: 2+3*4 -> 14; (2+3)*4 -> 20; 8-3-1 -> 4; -8/3 -> -2; 1/(2-2) -> division error at slash; empty/2 3/unknown name -> diagnostic; excess source, tokens, nodes or depth rejected.

Transfer reasoning: Put remainder at the multiplicative precedence level and choose a signed rule explicitly, such as a=b*q+r with q truncated toward zero and r carrying the dividend sign. Reject zero divisors; add lexer, parser, evaluator, bytecode, optimizer and negative-operand/error tests.

Lesson 15.3 — answers

15.3.Q1 — What is the stack-height change of a binary add that pushes one result?

Minus one: two values are removed and one is added.

15.3.Q2 — Why compare VM output to the tree evaluator?

Independent execution paths provide a useful differential check on compilation and interpretation.

15.3.Q3 — Does using bytecode automatically create a secure sandbox?

No. Host capabilities, memory safety, limits and implementation flaws still determine isolation.

15.3.Q4 — Why do subtraction tests matter even when addition tests pass?

Addition can conceal operand reversal; subtraction and division depend on the original left/right order.

15.3.Q5 — What does comparing two engines that share arithmetic validation fail to establish independently?

It cannot independently validate the shared arithmetic policy or implementation; separate golden and boundary tests are needed.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_15_3.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: 8-3*2 -> 2; five instructions pass with fuel 5, fail with 4; ADD on empty -> underflow; unknown opcode rejected; two final values rejected; PUSH beyond stack cap rejected; invalid literal and arithmetic range rejected.

Transfer reasoning: Validate jump targets and instruction-pointer changes, charge fuel for every executed instruction, analyze stack shapes at joins and loops, and bound storage independently. Backward jumps remove the baseline straight-line termination argument. Runtime fuel provides bounded interruption rather than proof of natural termination.

Lesson 15.4 — answers

15.4.Q1 — Does undecidability mean no individual program can be proven to terminate?

No. Many restricted or specific programs admit termination proofs.

15.4.Q2 — Why can floating-point reassociation change a result?

Rounding occurs after operations, so changing grouping can change which information is lost.

15.4.Q3 — What is the primary correctness condition for an optimization?

It must preserve the required observable behavior under the language and application assumptions.

15.4.Q4 — Why is replacing `(1/0)*0` by 0 invalid in this language?

The original must report division by zero. The rewrite removes a required observable failure.

15.4.Q5 — What structural facts let this tiny evaluator terminate without a general halting solver?

Accepted ASTs are finite, acyclic and depth/node bounded; generated bytecode is straight-line with bounded instructions and no unbounded host operations.

Lab review and independent transfer

The complete reference program is `code/reference/lesson_15_4.cpp`; inspect its supporting headers through CPP_REFERENCE.pdf's code map. Passing it is comparison evidence, not student completion. Check these required outcomes: empty digits rejected; 007 accepted; 7x rejected; 2+3*4 folds to 14; (1/0)*0* retains division error; range overflow preserved as error; left-before-right error location retained; optimized and unoptimized valid results agree.

Transfer reasoning: The two occurrences may have effects, throw errors or produce different values. Approve the rewrite only when the expression is pure, yields the same defined value on both evaluations, has no observable errors removed and obeys the numeric semantics. Floating-point NaN/infinity introduce additional restrictions.

Module 15 — exercise solutions

E15.1 — Type errors before execution

Problem: Write the rules for number addition and a Boolean condition. Classify 3+4, true+4 and if3then1else2 under your explicit toy rules.

Worked reasoning: Addition requires Number on both sides and returns Number; a condition requires Boolean and branches must satisfy the chosen result compatibility rule. 3+4 is valid; true+4 is invalid; using3 as a condition is invalid unless numeric truth conversion is an explicit language feature. These rules belong to the checker exercise; the bundled arithmetic evaluator does not implement if or Boolean literals.

E15.2 — Parse an associativity trap

Problem: Draw ASTs for 9-4-2 and 9-(4-2). Explain the parser control flow that produces left-associative subtraction.

Worked reasoning: The first AST is Sub(Sub(9,4),2) and yields3; the second is Sub(9,Sub(4,2)) and yields7. Parse an initial term, then loop over plus/minus, replacing the left subtree with a new binary node each time. A naive recursion on the entire remaining expression can unintentionally make subtraction right-associative.

E15.3 — Located diagnostics

Problem: For source 2 + (3 *), identify a useful error category and byte offset. What information should survive bytecode compilation?

Worked reasoning: With zero-based ASCII byte offsets, the unexpected right parenthesis is at9. Report a syntax error expecting an operand there, with source context. Bytecode instructions retain the originating operator/literal offset so a later runtime division error can identify its source. The exact wording is not fixed, but category, position and invariant preservation are testable.

E15.4 — Stack accounting

Problem: Trace PUSH8,PUSH3,SUB,PUSH2,MUL. Give stack depth after each instruction and the final value.

Worked reasoning: Stacks are[8],[8,3],[5],[5,2],[10], with depths1,2,1,2,1. A binary op removes right then left and pushes left op right. The maximum depth is2 and final depth1. Reversing operands would silently break subtraction and division even if addition tests passed.

E15.5 — A limit is not a sandbox

Problem: Your VM has 4,096 instructions of fuel. Explain why that alone does not bound lexing, parsing, allocation or a future host callback.

Worked reasoning: Fuel is one budget for the stage that charges it. Add source-byte, token, AST-node, nesting/depth, bytecode and stack limits before corresponding expensive work. A host callback may block or allocate beyond the VM's accounting. The supplied language has no I/O, pointers, jumps or callbacks, and is educational, not a general hostile-code security boundary.

E15.6 — Error-preserving optimization

Problem: Under the bounded integer language, may (1/0)*0 become0? May 2+3*4 become14? State a safe folding policy.

Worked reasoning: The first transformation removes a required division error and is forbidden. The second is valid because its checked operations all succeed with the same value. Fold a subtree only when checked evaluation succeeds; retain failing operations and source offsets. Evaluation order and side effects would require further reasoning if the language were extended.

E15.7 — Termination reasoning

Problem: Explain why a finite straight-line bytecode program terminates under the provided model, then why the same explanation fails after adding backward jumps.

Worked reasoning: Each executed instruction advances to the next finite position and each arithmetic operation is bounded, so the baseline completes or reports an error after at most its instruction count. Backward jumps break that decreasing-remaining-instructions argument. Fuel can force a controlled stop but does not establish that every unbounded program would naturally terminate.

E15.8 — Independent cross-check

Problem: A tree evaluator and VM both use the same checked arithmetic helper. What bugs can differential comparison miss, and how do you strengthen evidence?

Worked reasoning: A shared arithmetic bug can affect both implementations identically. Add hand-derived golden values, boundary/error cases and, where practical, a small independent mathematical model. Generated ASTs/expression strings should exercise precedence, associativity and malformed input, not only constant additions. Compare error category and location as well as successful values.

Module 15 — review answers

M15.R1 — Separate syntax, typing and execution with an example.

Syntax forms an AST; typing rejects disallowed operand categories; execution computes values or runtime errors. A syntactically valid division can still fail on zero.

M15.R2 — Why keep source offsets in tokens and instructions?

They connect parsing and runtime failures to original input; optimization must preserve the relevant origin for observable errors.

M15.R3 — Why reject final VM stack depth other than one?

The expression-program contract promises one result; zero or multiple leftovers indicate malformed/incomplete bytecode.

M15.R4 — What changes when you add host callbacks?

The trust boundary and resource model expand; instruction fuel alone does not bound callback time, memory or authority.

M15.R5 — Why is a matching optimized result on one expression insufficient?

Optimization must preserve all defined outcomes across its domain, including errors and their required context. Differential and golden tests provide evidence, not universal proof.

C-bridge answer key

C01

1. No. Language linkage does not make arbitrary C++ classes, templates or layouts into a portable C ABI. Use explicitly compatible declarations and document the toolchain boundary.
2. Capture the actual C compiler command and language mode, and inspect the C standard macro. The implementation also rejects __cplusplus. Filename alone is not proof.
3. The caller might read stale or partially changed output as a successful result. Specify unchanged, cleared or another deliberate policy and test it alongside status.

C02

1. The caller that accepts ownership must eventually call the matching destroy, normally through an owning wrapper. A borrower must not release it.
2. The C create/destroy pair defines the release operation. Default delete is not that pairing; a custom deleter invokes ge_device_destroy.
3. No. Opacity hides fields, not ownership aliases or erroneous repeated destroy calls. A unique-owner protocol and tests are still required.

C03

1. The implementation may insert padding to align members and the complete structure. The wire format is a separately defined field sequence.
2. Both functions could use the same wrong ordering. Compare encoded bytes with the independently specified golden sequence as well as round trips.
3. Return the documented error, do not interpret the payload under an incompatible schema, and preserve or clear output exactly as the API promises.

C04

1. A controlled hook targets a known path without destabilizing the system, competing with unrelated processes or relying on unpredictable allocation behavior.
2. No. One preserves prior data; the other explicitly clears an ownership output. Tests must enforce the function-specific promise.
3. Define synchronization, concurrent ownership/use rules and thread-safe failure instrumentation, then analyze races and test the new contract. The current global hooks are not thread-safe.

Sources and provenance

Continuity with the original program

This expansion was prepared for the September28,2026 curriculum edition. The original program manifest identifies Volume03 as C++, Algorithms and Language Implementation, Modules11–15, GateC and400–650 planning hours. Its lesson IDs, titles, outcomes, prerequisites, practical assignments and assigned readings are preserved in `source/ORIGINAL_VOLUME_03_MANIFEST.json`. The 60 original quiz questions and answers were recovered from the original `CURRICULUM.html`; the expansion adds40 application questions without replacing the originals.

Teaching explanations, worked examples, code, additional exercises, reviews, bridge material and project elaborations are original instructional synthesis for this bundle. Primary reading references support the underlying concepts. No external textbook, paper, compiler distribution, proprietary SDK or font file is included. Linked resources retain their own terms. Current documentation was consulted on September28,2026; historical course archives are intentionally used for durable foundations. Not every section of every historical source was re-reviewed in full.

The original assignments are preserved even where the starter supplies only a first contract. The full lab remains a learner requirement. Complete reference examples provide comparison evidence, not pre-completed module projects. The source JSON distinguishes original questions from additions.

Primary reference directory

R06 — 6.042J Mathematics for Computer Science

MIT OpenCourseWare. Proofs, graphs, counting and discrete probability.

https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/

R11 — C++ Core Guidelines

C++ Core Guidelines editors. Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

R12 — CMake tutorial

Kitware. Build systems; use documentation matching the installed release.

https://cmake.org/cmake/help/latest/guide/tutorial/index.html

R13 — 6.006 Introduction to Algorithms

MIT OpenCourseWare. Data structures, graph algorithms and dynamic programming.

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/

R14 — Kaleidoscope and LLVM tutorials

LLVM Project. Compiler frontends, intermediate representation and JIT compilation.

https://llvm.org/docs/tutorial/

R15 — 6.004 Computation Structures

MIT OpenCourseWare. Logic, processor design, memory and systems.

https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/

R16 — Operating Systems: Three Easy Pieces

Remzi and Andrea Arpaci-Dusseau. Virtualization, concurrency and persistence; book by its authors.

https://pages.cs.wisc.edu/~remzi/OSTEP/

R20 — 15-466/666 Computer Game Programming

Carnegie Mellon University. Runtime systems and game-production programming.

https://graphics.cs.cmu.edu/courses/15-466/

P4-01 — CMake compile features and language standards

Kitware. Target requirements, language standards and extensions. Use the documentation matching your CMake release.

https://cmake.org/cmake/help/latest/manual/cmake-compile-features.7.html

P4-02 — What Every Computer Scientist Should Know About Floating-Point Arithmetic

David Goldberg / Oracle documentation. Author paper reproduced in official numerical-computation documentation; rounding and non-associativity background.

https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html

P4-03 — AddressSanitizer

LLVM / Clang. Compiler/runtime setup and kinds of executed memory errors detected. Detection is not an all-input proof.

https://clang.llvm.org/docs/AddressSanitizer.html

P4-04 — UndefinedBehaviorSanitizer

LLVM / Clang. Runtime instrumentation and supported checks. Platform/toolchain support must be verified.

https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

P4-05 — Language standards supported by GCC

GNU Compiler Collection. C/C++ standards and language modes. This course selects C17/C++20, not a newest-standard claim.

https://gcc.gnu.org/onlinedocs/gcc/Standards.html

P4-06 — Kaleidoscope: implementing a parser and AST

LLVM Project. Frontend architecture reading; the course language and integer semantics are independently specified and do not require LLVM libraries.

https://llvm.org/docs/tutorial/MyFirstLanguageFrontend/LangImpl02.html

P4-07 — Fundamental types [basic.fundamental]

C++ working-draft editors / eel.is rendering. Current working-draft view, not the fixed published C++20 edition. Consult only the stable integer/representation concepts used here; later additions are outside this course baseline.

https://eel.is/c++draft/basic.fundamental

P4-08 — Vector modifiers, capacity and unordered-container requirements

C++ working-draft editors / eel.is rendering. Companion sections: https://eel.is/c++draft/vector.capacity and https://eel.is/c++draft/unord.req. Used for operation-specific invalidation rules; current draft contains later features not used by the course.

https://eel.is/c++draft/vector.modifiers

Reading by topic

Use R11 for ownership, interfaces, special members, exception safety and source-file organization; R12/P4-01 for builds; R13 for arrays, hashing, heaps, sorting, graphs and dynamic programming; R14/P4-06 for language frontends and compiler structure. R06 supports discrete proof and computability reasoning, and R15 connects representations to later hardware study. R16/R20 are cross-volume reading pointers, not dependencies of the supplied build.

Read the exact assignment printed under each lesson. Record the section actually studied and at least one exercise or derivation in the journal. The narrow course implementation deliberately differs from some reading examples: bounded integers instead of general floating-point language values, a small straight-line stack machine instead of an LLVM JIT, and mock resources instead of a graphics API. Do not merge incompatible assumptions silently.